Escola de Artes, Ciências e Humanidades — Universidade de São Paulo

PROJETO DE INICIAÇÃO CIENTÍFICA

Resiliência do ASCON: Infraestrutura QEMU/GDB e Scripts para Injeção Automatizada de Falhas

Área: Sistemas de Informação / Segurança da Informação
Estudante: Eduardo Diniz da Silva
Orientador: Prof. Fábio Nakano

1. Resumo

Este projeto de iniciação científica propõe avaliar quantitativamente a resiliência da família de algoritmos de criptografia leve ASCON frente a injeções de falhas lógicas em microcontroladores, utilizando um ambiente de execução simulado. Em dispositivos de borda e da Internet das Coisas (IoT) desprovidos de Unidades de Gerenciamento de Memória (MMU), a proteção contra perturbações lógicas em tempo de execução torna-se crítica. A metodologia divide-se em quatro fases: fundamentação teórica do padrão ASCON e formalização do modelo de falhas; configuração da infraestrutura de emulação Linux baseada em QEMU e depuração via GDB; desenvolvimento de scripts para injeções automatizadas na memória RAM e nos registradores; e aplicação de técnicas de tolerância a falhas via software (SIHFT e CFC) para endurecimento (hardening) do código C oficial. Espera-se catalogar as vulnerabilidades do algoritmo sob o modelo SEU (Single Event Upset), quantificar os efeitos de degradação e disponibilizar uma implementação otimizada e de código aberto. A pesquisa contribui para a literatura de criptografia leve e fortalece o ecossistema de Tecnologias Sociais ao democratizar o acesso a soluções embarcadas seguras para aplicações comunitárias.

Palavras-chave: Microcontroladores; Criptografia Leve; Família ASCON; Injeção de Falhas; Tolerância a Falhas.

2. Introdução e Justificativa

Com o avanço da internet, as conexões deixaram de ser exclusivamente entre usuários e passaram a integrar a comunicação máquina a máquina (M2M). Atualmente, a expansão massiva da Internet das Coisas (IoT) ocorre fortemente condicionada ao baixo custo dos componentes, com projeção de atingir 40 bilhões de conexões até 2034 (Statista; Transforma Insights, 2024), tornando-se uma das bases técnicas da conectividade contemporânea em cenários que vão de residências a indústrias e infraestruturas críticas. Contudo, para atingir tal viabilidade econômica, os fabricantes empregam microcontroladores e System-on-Chips (SoCs) dotados de pouca memória, baixo clock e limitação energética (Lima; Viana, 2026). A

negligência com a segurança em prol da contenção de custos resultou em incidentes graves de escala global, como os ataques massivos da botnet Mirai, que comprometeu milhares de dispositivos de borda vulneráveis e paralisou serviços essenciais na internet (Antonakakis et al., 2017; Lima; Viana, 2026).

Em decorrência dessas limitações de custo e arquitetura, a maioria dos microcontroladores e SoCs de entrada não contêm Unidades de Gerenciamento de Memória (MMU) (Lima; Viana, 2026). Sem o isolamento físico de hardware, tarefas e threads executadas simultaneamente compartilham o mesmo espaço de endereçamento, criando o risco de leitura não autorizada ou adulteração de buffers de memória por processos concorrentes (Lima; Viana, 2026). É nesse contexto que se insere o ASCON, selecionado pelo National Institute of Standards and Technology (NIST) como o novo padrão mundial de Criptografia Leve (Alcântara; Okano, 2023). Embora o ASCON assegure a confidencialidade e a autenticidade de dados em trânsito e em repouso via Cifragem Autenticada com Dados Associados (AEAD) (Alcântara; Okano, 2023), ele não supre a ausência de MMU para o isolamento de threads na RAM. Como chaves simétricas e dados em texto plano coexistem na memória compartilhada durante o processamento, a segurança do sistema contra distúrbios e execuções indevidas passa a depender do endurecimento do próprio algoritmo e da avaliação de sua resiliência em tempo de execução (Elad et al., 2026; Halderman et al., 2009).

Adicionalmente, uma das vulnerabilidades críticas em microcontroladores de baixo custo é a exposição de interfaces físicas de depuração, como JTAG, UART e SWD, as quais possibilitam a leitura não autorizada da memória Flash, engenharia reversa do firmware, extração de segredos e clonagem de dispositivos (Lima; Viana, 2026). É importante ressaltar que o algoritmo ASCON não impede o acesso físico ou a extração de dados por meio dessas portas. Contudo, a aplicação da cifragem protege o conteúdo sensível armazenado na memória Flash ao mantê-lo autenticado e ilegível, garantindo que um eventual dump de memória revelará apenas dados ininteligíveis aos atacantes. No entanto, a ameaça aos dispositivos embarcados não se limita à extração passiva. A integridade operacional do sistema também pode ser severamente comprometida por ataques de Injeção de Falhas (Fault Injection Attacks – FIA), caracterizados por perturbações lógicas deliberadas que provocam a alteração de valores em registradores ou posições de memória durante a execução do algoritmo (Dantas, 2023). Torna-se fundamental, portanto, avaliar como a execução do ASCON se comporta frente a tais distúrbios lógicos.

Para a realização deste estudo, adota-se a implementação oficial de referência em linguagem C do algoritmo ASCON, disponibilizada em código aberto por seus criadores (ASCON Team, 2026). Em vez de bancadas físicas complexas, o experimento é conduzido em um ambiente simulado em sistema Linux apoiado no emulador de arquiteturas QEMU em conjunto com o depurador GDB, uma abordagem consagrada na literatura para testes de confiabilidade e injeção de falhas por software (Geissler, 2014; Cucchiara, 2025). Por meio dessa infraestrutura, scripts customizados interceptam a execução do ASCON em tempo de execução para injetar falhas lógicas deliberadas na memória RAM e nos registradores da CPU emulada (Geissler, 2014). Esse arranjo experimental possibilita o controle total sobre o momento e o modelo da injeção, permitindo avaliar quantitativamente a resiliência do código e mapear o grau de degradação da cifra sob perturbações lógicas.

Para mensurar o impacto dessas perturbações na execução do algoritmo, os scripts de injeção elaborados neste estudo categorizarão os efeitos com base no modelo de degradação em registradores por Single Event Upset (SEU) (Dantas, 2023) e na taxonomia de falhas estabelecida na literatura (Geissler, 2014; Sini; Solouki; Violante, 2023). As ocorrências serão classificadas em Corrupção Silenciosa de Dados (Silent Data Corruption – SDC), Falhas Detectadas e Erros de Fluxo de Controle (Control Flow Errors – CFE), decorrentes de falhas aleatórias de hardware (Random Hardware Failures – RHF) (Geissler, 2014; Sini; Solouki; Violante, 2023). A avaliação de técnicas de tolerância a falhas via software (SIHFT) e de checagem do fluxo de controle (CFC) diretamente em linguagem C assegura a independência de arquitetura e a portabilidade do código. Essa análise preventiva consolida o princípio de Security by Design no desenvolvimento do firmware, permitindo incorporar requisitos mínimos de resiliência desde as fases iniciais do projeto sem elevar os custos de fabricação do hardware (Lima; Viana, 2026; Sini; Solouki; Violante, 2023). Dessa forma, as diretrizes de otimização geradas não apenas enriquecem a literatura sobre criptografia leve, mas também fortalecem o ecossistema de Tecnologias Sociais, democratizando o acesso a soluções embarcadas seguras e de código aberto para aplicações de interesse comunitário (Dagnino, 2014).

3. Objetivos

3.1 Objetivo Geral

Avaliar o comportamento de uma implementação em C da família de algoritmos ASCON a injeções de falhas lógicas deliberadas em um ambiente simulado apoiado no emulador de arquiteturas QEMU em conjunto com o depurador GDB.

3.2 Objetivos Específicos

- Estudar a base teórica da criptografia leve ASCON.
- Levantar modelos de ataque por software.
- Configurar o ambiente de emulação e depuração.
- Compilar a implementação oficial em C do ASCON no ambiente simulado.
- Desenvolver scripts automatizados de injeção de falhas.
- Executar testes massivos de injeção de falhas.
- Categorizar e quantificar o impacto.
- Implementar técnicas de tolerância a falhas.
- Avaliar os resultados.
- Redigir um relatório final do estudo.
- Disponibilizar o código C otimizado.

4. Metodologia

O estudo será desenvolvido em quatro fases sequenciais ao longo de 12 meses através de estudos teóricos, aplicação do algoritmo, simulação de falhas, avaliação empírica dos resultados e otimização do algoritmo.

Fase 1 — Fundamentação Teórica e Preparação do Ambiente (meses M1–M3)

- Estudar a especificação do algoritmo ASCON (cifragem AEAD, funções de permutação e tamanho de estado).
- Configurar o ambiente simulado Linux utilizando QEMU para a emulação do microcontrolador alvo e integração com o depurador GDB.
- Formalizar o rol metodológico de falhas, mapeando a taxonomia da literatura (SEU, SDC, CFE e RHF) que orientará a construção dos testes.
- Produzir um relatório de revisão bibliográfica contendo a arquitetura do ambiente simulado e a matriz de classificação de falhas configurada.

Fase 2 — Compilação do código e Automação de Scripts (meses M4–M6)

- Compilar e executar a implementação de referência oficial do Ascon em C no ecossistema QEMU/GDB.
- Coletar métricas padrão de desempenho de execução.
- Desenvolver scripts automatizados (em Python/GDB) para a manipulação da memória RAM e dos registradores da CPU emulada em tempo de execução.

Fase 3 — Campanha Experimental de Testes e Análise (meses M7–M9)

- Executar rodadas experimentais de injeção de falhas interceptando a permutação interna do ASCON.
- Classificar os efeitos observados nas categorias predefinidas: Corrupção Silenciosa de Dados (SDC), Erros de Fluxo de Controle (CFE) e Exceções/Paradas do Sistema.
- Mapear quais posições de registradores ou variáveis de estado apresentam maior vulnerabilidade a alterações lógicas transientes.
- Construir um banco de dados quantitativo com os resultados dos testes e gráficos de propagação e severidade de falhas do Ascon.

Fase 4 — Hardening de Software, Otimização e Conclusão (meses M10–M12)

- Aplicar técnicas de Software-Implemented Hardware Fault Tolerance (SIHFT) e Checagem de Fluxo de Controle (CFC) diretamente no código C para mitigar as falhas mais críticas encontradas na Fase 3.
- Reavaliar o código modificado no ambiente simulado, medindo o ganho de resiliência em relação ao aumento no uso de memória e no tempo de processamento.
- Redigir o Relatório Final da Iniciação Científica e submeter o trabalho a eventos acadêmicos.
- Disponibilizar código C otimizado com rotinas de tolerância a falhas.

5. Plano de Trabalho e Cronograma

Atividade	**M1**	**M2**	**M3**	**M4**	**M5**	**M6**	**M7**	**M8**	**M9**	**M10**	**M11**	**M12**
Estudo Algoritmo	X	X										
Levantamento de Modelos		X	X									

Configuração do Ambiente	X	X									
Formalização das Falhas		X									
Produção do Relatório		X									
Implementação do Código			X								
Coleta de Métricas			X	X							
Desenvolvimento dos Scripts				X	X						
Experimentação da Injeção						X	X				
Classificar os Efeitos							X	X			
Mapeamento de Vulnerabilidades							X	X			
Construir Banco de Dados								X			
Aplicação do SIHFT e CFC									X	X	
Reavaliação do Código										X	X
Produção do Relatório Final											X
Disponibilização do Código											X

---

6. Resultados Esperados

- Infraestrutura em ambiente Linux integrando o emulador QEMU com o depurador GDB configurada e automatizada para emulação e testes.
- Conjuntos de scripts para interceptação em tempo de execução e injeção de falhas desenvolvido.
- Mapeamento quantitativo de resiliência e taxonomia de falhas.
- Implementação do código ASCON otimizado e com a aplicação de técnicas de tolerância a falhas via software e checagem do fluxo de controle.
- Código otimizado e scripts de simulação publicados em um repositório aberto.

- Artigo científico submetido e resultados apresentados em congressos.

7. Referências

ALCÂNTARA, Gabriel Barbosa; OKANO, Gabriel Hirofumi. Estudo e Implementação do Novo Padrão de Criptografia Leve ASCON. 2023. Trabalho de Conclusão de Curso (Graduação em Engenharia de Computação) – Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, 2023.

ANTONAKAKIS, Manos et al. Understanding the Mirai Botnet. In: USENIX SECURITY SYMPOSIUM, 26., 2017, Vancouver. Proceedings... Vancouver: USENIX Association, 2017. p. 1093-1110.

ASCON TEAM. Ascon Implementations. Graz: Graz University of Technology, 2026. Disponível em: https://ascon.isec.tugraz.at/implementations.html. Acesso em: 6 ago. 2026.

CUCCHIARA, Luana. Rust4Safety: Comparison of Software-Implemented Hardware Fault Tolerance Techniques between C and Rust Programming Languages. 2025. Dissertação (Mestrado em Engenharia de Computação) – Politecnico di Torino, Turim, 2025.

DAGNINO, Renato. Tecnologia Social: contribuições conceituais e metodológicas. Campina Grande: EDUEPB, 2014. 318 p. ISBN 978-85-7879-327-2. Disponível em: http://books.scielo.org. Acesso em: 6 ago. 2026.

DANTAS, Matheus Luna de Oliveira. Injeção de falhas e verificação de hardware para o CEVERO. 2023. Trabalho de Conclusão de Curso (Graduação em Engenharia de Computação) – Escola Politécnica, Universidade de São Paulo, São Paulo, 2023.

ELAD, Georgette Jocelyne et al. AI-Driven Adaptive Lightweight Cryptography for IoT Healthcare Systems: A Contextual Security Framework for Resource-Constrained Environments. Journal of Information Security, [S. l.], v. 17, n. 2, 18 mar. 2026.

GEISSLER, Filipe de Aguiar. Metodologia de injeção de falhas baseada em emulação de processadores. 2014. Dissertação (Mestrado em Engenharia Elétrica) – Escola de Engenharia, Universidade Federal do Rio Grande do Sul, Porto Alegre, 2014.

HALDERMAN, J. Alex et al. Lest We Remember: Cold Boot Attacks on Encryption Keys. Communications of the ACM, New York, v. 52, n. 5, p. 91-98, mai. 2009.

LIMA, Gleibson da Silva; VIANA, Paulino Wagner Palheta. Segurança em IoT de baixo custo: vulnerabilidades, matriz de riscos e diretrizes de hardening para sistemas embarcados. Revista FT, v. 30, n. 159, p. 01-11, 2026.

SINI, Jacopo; SOLOUKI, Mohammadreza Amel; VIOLANTE, Massimo. Guidelines for implementing control flow checking into automotive embedded applications developed with

C language. In: IEEE NORDIC CIRCUITS AND SYSTEMS CONFERENCE (NorCAS), 2023, Aalborg. Proceedings... Piscataway: IEEE, 2023. p. 1-7.

STATISTA; TRANSFORMA INSIGHTS. Number of IoT connected devices worldwide from 2019 to 2030. Statista, 2024. Disponível em: https://www.statista.com/statistics/1183457/iot-connected-devices-worldwide/. Acesso em: 6 ago. 2026.